TCS NQT · ADVANCED CODING

# Advanced Coding Zero to Hero

Every pattern behind the TCS coding problems: arrays, strings, math, greedy, DP, backtracking, grids, graphs, bits. Working code in C, C++, Java and Python.

| 13 | 41 | 86 | 54 |
|---|---|---|---|
| chapters, zero to hero | solved examples | practice questions | diagrams and charts |

ONLINE LEARNING MOCK TEST SERIES · ATC DEVELOPERS

# About this guide

This guide teaches the patterns that sit behind TCS Advanced Coding problems, from reading the constraint to the final edge-case check. Every program shown was compiled and run while the guide was built, and each algorithm was compared with a brute-force solution on random small inputs. The chapter on real TCS problems studies seven actual problems.

### How each chapter is built

- **Idea** in plain words, then a diagram.
- **Formulas and rules** in the yellow box.
- **How the questions appear** in the exam.
- **Solved examples** with full steps.
- **Shortcuts and mistakes** to avoid.
- **Practice set** (Basic, Medium, Advanced) with answers at the end.

### How to study

- Type every program yourself in the language you will use in the exam.
- After each chapter solve its practice set, then the matching Practice Arena problems.
- Learn to read the constraint first: it tells you the algorithm.
- Finish with the real problems chapter and the three-problem mini test under a 90-minute timer.

## Contents


1. How TCS Advanced Coding Works — Foundation — 3
2. Arrays: Prefix Sums, Windows — Must-do — 7
3. Strings and Hashing — Must-do — 12
4. Sorting, Binary Search, Greedy — Must-do — 16
5. Math and Number Theory — Must-do — 20
6. Dynamic Programming — Advanced — 24
7. Recursion and Backtracking — Important — 29
8. Stacks and Queues — Important — 33
9. Grids: BFS, DFS, Matrices — Must-do — 37
10. Graphs, Trees, Linked Lists, DSU — Advanced — 42
11. Bit Manipulation — Important — 46
12. The Real TCS Problems — Must-do — 50
13. Debugging, Mini Test, Cheat Sheet — Final — 58

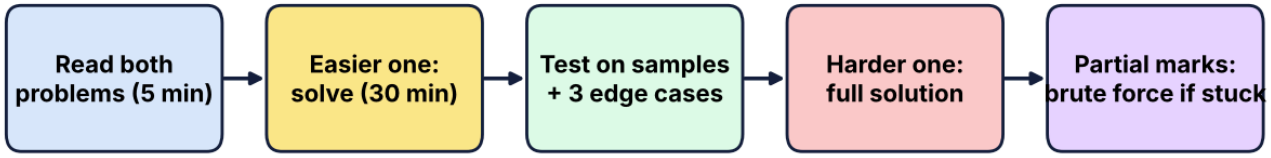

### Honest note

- Every number in the solved examples and in the practice answers was calculated by a program while this guide was made, and puzzle answers were found by exhaustive search where possible.
- TCS does not publish its question weightage. Priority labels are our preparation guidance.

# 01 ADVANCED CODING · START HERE
# How TCS Advanced Coding Works

Advanced Coding is the section that decides Digital and Prime. In the pattern used in this series it has **two problems in about 90 minutes**, solved in C, C++, Java or Python, and marked by **hidden test cases**. A program that is correct on the samples but slow on large inputs, or wrong on one edge case, loses marks. This guide teaches the patterns behind the problems, with code that was **compiled and run** while the guide was built. Check the official TCS portal for the latest pattern.

**A 90-minute plan for two coding problems**




Take the easier problem first; keep 10 minutes for edge cases.

## The four things the judge checks

| Check | Meaning | How to pass |
|---|---|---|
| Correct output | Exact text, spaces and newlines | Print exactly what is asked; no extra prompts |
| Time limit | Usually 1–2 seconds | Choose algorithm by constraint |
| Memory limit | Typically 256 MB | Avoid huge 2-D arrays when n = $10^5$ |
| Hidden cases | Edge and large cases | n = 1, all equal, negative numbers, maximum n |

**Read the constraint, pick the algorithm**

| Constraint | Allowed complexity | Typical technique |
|---|---|---|
| **n ≤ 10** | O(n!) or O($2^n$·n) | brute force, backtracking |
| **n ≤ 20** | O($2^n$) | bitmask, subsets |
| **n ≤ 500** | O($n^3$) | three loops, Floyd–Warshall |
| **n ≤ 5,000** | O($n^2$) | nested loops, 2-D DP |
| **n ≤ $10^5$** | O(n log n) | sort, binary search, heap |
| **n ≤ $10^6$** | O(n) | one pass, prefix sums, hashing |
| **n ≤ $10^{18}$** | O(log n) | math, fast power, gcd |

About $10^8$ simple operations fit in one second.

## Input and output templates

Every problem starts the same way: read n, read n numbers, compute, print. Here is the same program (sum and maximum) in all four languages. Each one was compiled and run on the same input and all four printed the same answer.

**Python 3**

```
n = int(input())
a = list(map(int, input().split()))
print(sum(a), max(a))
```

**Input**

```
5
3 -1 4 1 5
```

**Output** (run and checked)

```
12 5
```

**C++17**

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n; cin >> n;
    vector<long long> a(n);
    for (auto &x : a) cin >> x;
    long long s = 0, mx = a[0];
    for (auto x : a) { s += x; mx = max(mx, x); }
    cout << s << " " << mx << endl;
}
```

**Input**

```
5
3 -1 4 1 5
```

**Output** (run and checked)

```
12 5
```

**C**

```
#include <stdio.h>
int main() {
    int n; scanf("%d", &n);
    long long s = 0, mx = -1000000000000LL, x;
    for (int i = 0; i < n; i++) { scanf("%lld", &x); s += x; if (x > mx) mx = x; }
    printf("%lld %lld\n", s, mx);
    return 0;
}
```

**Input**

```
5
3 -1 4 1 5
```

**Output** (run and checked)

```
12 5
```

Java

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        long s = 0, mx = Long.MIN_VALUE;
        for (int i = 0; i < n; i++) { long x = sc.nextLong(); s += x; mx = Math.max(mx, x); }
        System.out.println(s + " " + mx);
    }
}
```

**Input**

```
5
3 -1 4 1 5
```

**Output** (run and checked)

```
12 5
```

**What does each of the four programs print for the input 5 / 3 −1 4 1 5?**

**12 5**

Sum = 3 − 1 + 4 + 1 + 5 = 12; maximum = 5. All four were executed and returned the same text.

### Language traps

- **C / C++:** use `long long` when sums can pass $2^{31} - 1$ (about $2.1 \times 10^9$). Overflow is silent.
- **C++:** add `ios::sync_with_stdio(false); cin.tie(nullptr);` for $10^5$ or more numbers.
- **Java:** `Scanner` is slow for big input; use `BufferedReader` with `StreamTokenizer` or split on spaces.
- **Python:** use `sys.stdin.readline` or read all at once for large input; recursion depth is 1000 by default.
- **All:** integer division truncates toward zero in C/C++/Java but floors in Python for negatives.

## Reading TCS problem statements

| Story words | Usually means | Chapter |
|---|---|---|
| “students stand in a row … each moves to index B[i]” | Permutation cycles, LCM | Math |
| “minimum number of moves/coins” | Greedy or DP | Greedy, DP |
| “number of subarrays with sum K” | Prefix sum + hash map | Arrays |
| “rooms / houses in a line” | Array problem | Arrays |
| “minimum letters / convert string A to B” | String comparison, greedy | Strings |
| “grid / maze / islands” | BFS or DFS | Grids |
| “all possible ways” | DP counting or backtracking | DP, Recursion |

### Shortcuts and tricks

- Write the constraint on paper first; it tells you the target complexity.
- Always test: n = 1, n = 2, all equal, already sorted, maximum size.
- Print the answer with a newline only. Do not print “Enter n”.
- If stuck, submit a brute-force solution: it earns marks on small tests.

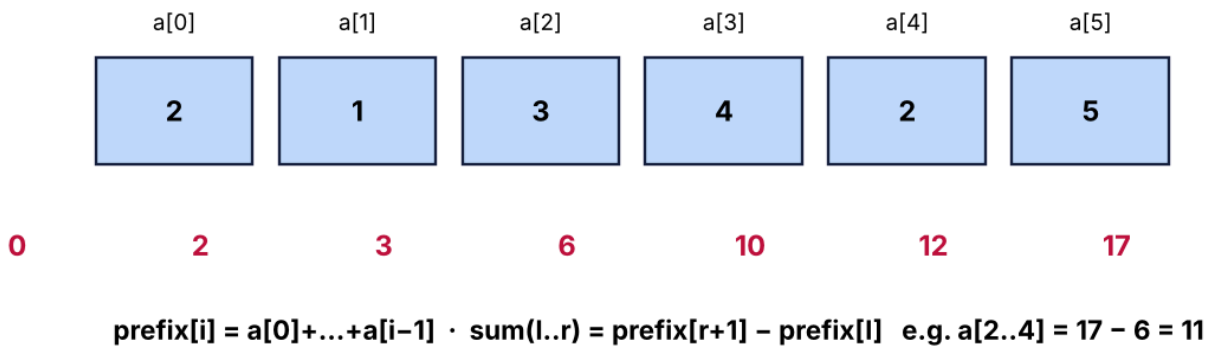
### Common mistakes

- Using `int` where the sum needs `long long`.
- Reading the input in the wrong order.
- Printing extra spaces or text.
- Spending 60 minutes on one problem.

ADVANCED CODING · ARRAYS · MUST-DO

# 02 Arrays: Prefix Sums, Two Pointers, Sliding Window

Arrays are the most common topic: roughly half of all coding problems reduce to a running sum, two moving indices, or a hash map of what you have seen. If a problem says **“subarray”, “contiguous”, “pairs”, “longest/shortest segment”**, start here.




Prefix sums turn any range sum into one subtraction.

### Patterns

- **Prefix sum:** sum(l..r) = prefix[r+1] − prefix[l]. O(n) to build, O(1) per query.
- **Prefix + hash map:** to count subarrays with sum K, look up how many earlier prefixes equal (current − K).
- **Kadane:** best subarray ending here = max(x, previous + x).
- **Two pointers (sorted):** move i up if the sum is too small, j down if too large.
- **Sliding window:** grow right end, shrink left end when the condition breaks. O(n).
- **Difference array:** add v to l..r in O(1), rebuild with one pass.

## 1. Maximum subarray sum (Kadane)

Given an array with negative numbers, find the largest sum of a contiguous subarray. Brute force is $O(n^2)$; Kadane is O(n).

Python 3

```python
def max_subarray(a):
    best = cur = a[0]
    for x in a[1:]:
        cur = max(x, cur + x)      # extend or restart
        best = max(best, cur)
    return best

n = int(input())
a = list(map(int, input().split()))
print(max_subarray(a))
```

**Input**

```
9
-2 1 -3 4 -1 2 1 -5 4
```

**Output** (run and checked)

```
6
```

**Maximum subarray sum of −2 1 −3 4 −1 2 1 −5 4.**

**6**

The best segment is 4, −1, 2, 1 with sum 6. The code was checked against a brute-force search on 300 random arrays.

## 2. Subarrays with sum K (prefix sums + hash map)

Count the contiguous subarrays whose sum is exactly K. A TCS problem (“rooms with exactly K coins”) is this pattern.

C++17

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n; long long k; cin >> n >> k;
    unordered_map<long long,long long> seen; seen[0] = 1;
    long long pre = 0, ans = 0;
    for (int i = 0; i < n; i++) {
        long long x; cin >> x; pre += x;
        if (seen.count(pre - k)) ans += seen[pre - k];
        seen[pre]++;
    }
    cout << ans << endl;
}
```

**Input**

```
5 7
3 4 7 2 -3
```

**Output** (run and checked)

```
2
```

**Count subarrays of [3, 4, 7, 2, −3] with sum 7.**

**2**

The segments are [3,4] and [7]. The C++ program above printed 2, and the Python version was compared with brute force on 300 random arrays.

## 3. Longest substring without repeating characters (sliding window)

**Python 3**

```
def longest_unique(s):
    last = {}
    start = best = 0
    for i, ch in enumerate(s):
        if ch in last and last[ch] >= start:
            start = last[ch] + 1   # shrink window
        last[ch] = i
        best = max(best, i - start + 1)
    return best

print(longest_unique(input().strip()))
```

**Input**

```
abcabcbb
```

**Output** (run and checked)

```
3
```

**Longest substring without repeating letters in “abcabcbb”.**

**3**

“abc” has length 3. The left end jumps just past the previous copy of a repeated letter.

## 4. Pairs with a given sum (two pointers)

Python 3

```
def pairs_with_sum(a, k):
    a = sorted(a)
    i, j, cnt = 0, len(a) - 1, 0
    while i < j:
        s = a[i] + a[j]
        if s == k:
            cnt += 1; i += 1; j -= 1
        elif s < k:
            i += 1
        else:
            j -= 1
    return cnt

k = int(input())
a = list(map(int, input().split()))
print(pairs_with_sum(a, k))
```

**Input**

```
10
1 2 3 4 5 6 7 8 9
```

**Output** (run and checked)

```
4
```

**Pairs of distinct values in 1..9 with sum 10.**

**4**

(1,9), (2,8), (3,7), (4,6). Sort first; two pointers finish in O(n log n).

| Wording | Pattern | Cost |
|---|---|---|
| “largest sum of a contiguous part” | Kadane | O(n) |
| “number of subarrays with sum K” | Prefix + dict | O(n) |
| “longest substring with …” | Sliding window | O(n) |
| “pair with sum K” (sorted) | Two pointers | O(n) |
| “range sum, many queries” | Prefix array | O(1) each |
| “add v to l..r, many times” | Difference array | O(1) each |

### Shortcuts and tricks

- If negative numbers are present, sliding window for “sum = K” fails: use prefix + hash map.
- Initialise the hash map with {0: 1} so subarrays starting at index 0 are counted.
- Use 64-bit sums in C/C++/Java.

### Common mistakes

- Using a sliding window on arrays that contain negatives.
- Forgetting {0: 1} in the prefix map.
- Using Kadane with the initial best = 0 (wrong when all numbers are negative).

## Practice set: Array practice

| | | |
|---|---|---|
| **1.** | Max subarray sum of [5, −9, 6, −2, 3]? | Basic |
| **2.** | Max subarray sum of [−3, −1, −2]? | Basic |
| **3.** | Subarrays of [1, 1, 1] with sum 2? | Medium |
| **4.** | Subarrays of [1, 2, 3, −3, 3] with sum 3? | Medium |
| **5.** | Longest substring without repeats in “pwwkew”? | Medium |
| **6.** | Longest substring without repeats in “abba”? | Advanced |
| **7.** | Pairs summing to 8 in [1,2,3,4,5,6,7]? | Advanced |
| **8.** | Subarrays of [0, 0, 0, 0] with sum 0? | Advanced |

# 03 Strings and Hashing

ADVANCED CODING · STRINGS · MUST-DO

String problems check three skills: **counting characters** (a frequency table), **comparing strings** (sorting letters, two pointers) and **scanning with a counter** (brackets). Most are solved in one pass with a dictionary or a counter.

### Patterns

- **Frequency table:** `Counter(s)` or an array of 26 counts. Two strings are anagrams if their counts match.
- **Canonical form:** sort the letters; words with the same sorted string are anagrams.
- **Palindrome:** expand around each centre (odd and even). $O(n^2)$ is fine for $n \le 5{,}000$.
- **Brackets:** keep a counter; never let it go below 0; unmatched closings are added directly.
- **Window / last seen index:** longest substring without repeats (Chapter 2).
- **Case and spaces:** read the statement; "ignore case" means lower() both sides.

## 1. Longest palindromic substring

Python 3

```
def longest_pal(s):
    best = ""
    for c in range(len(s)):
        for l, r in ((c, c), (c, c + 1)):      # odd and even centres
            while l >= 0 and r < len(s) and s[l] == s[r]:
                l -= 1; r += 1
            if r - l - 1 > len(best):
                best = s[l + 1:r]
    return best

print(longest_pal(input().strip()))
```

**Input**

```
forgeeksskeegfor
```

**Output** (run and checked)

```
geeksskeeg
```

**Longest palindromic substring of "forgeeksskeegfor".**

**geeksskeeg**

Expand from each centre; the even centre between the two "s" gives "geeksskeeg" (length 10). The code was verified against brute force on 150 random strings.

## 2. Count anagram groups

Java

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        Map<String, Integer> g = new HashMap<>();
        for (int i = 0; i < n; i++) {
            char[] c = sc.next().toCharArray();
            Arrays.sort(c);
            g.merge(new String(c), 1, Integer::sum);
        }
        System.out.println(g.size());
    }
}
```

**Input**

```
6
eat tea tan ate nat bat
```

**Output** (run and checked)

```
3
```

**How many anagram groups are in: eat, tea, tan, ate, nat, bat?**

**3**

{eat, tea, ate}, {tan, nat}, {bat}: 3 groups. The Java program above compiled and printed 3; the Python version gives the same.

## 3. Minimum brackets to add

Python 3

```
def min_add(s):
    open_need = add = 0
    for ch in s:
        if ch == "(":
            open_need += 1
        elif open_need:
            open_need -= 1
        else:
            add += 1                # unmatched ")"
    return add + open_need

print(min_add(input().strip()))
```

**Input**

```
()))(
```

**Output** (run and checked)

```
3
```

**Minimum brackets to add to make "())((" balanced.**

**3**

The first ")" matches, the second ")" is unmatched (+1), then two "(" remain open (+2). Total 3. Checked against an exhaustive insertion search on 60 random strings.

## 4. Convert string A to B using "replace with the smallest letter" moves

This is the idea of a real TCS Advanced Coding problem. A move chooses a set of positions and replaces all of their letters by the smallest letter among them. Letters can only go down.

**Python 3**

```
def min_moves(A, B):
    if any(a < b for a, b in zip(A, B)):
        return -1
    need = {b for a, b in zip(A, B) if a > b}
    if any(t not in set(A) for t in need):
        return -1
    return len(need)

A = input().strip(); B = input().strip()
print(min_moves(A, B))
```

**Input**

```
abcab
aabab
```

**Output** (run and checked)

```
2
```

**Minimum moves to turn "abcab" into "aabab", and "de" into "cd".**

**2; -1**

For abcab → aabab, position 2 needs b→a and position 3 needs c→b. Each distinct target letter that must come down costs one move (here a and b), and both letters exist in A, so the answer is 2. "de" → "cd" is impossible because "d" cannot become "c" when no "c" exists in A. This rule was checked against a brute-force search over all moves on 400 random small strings (earlier verification, see the Actual Questions PDF).

| Wording | Technique | Time |
|---|---|---|
| "anagram" | Sorted key or counts | O(n log n) |
| "palindrome" | Expand around centre | $O(n^2)$ |
| "balanced brackets" | Counter or stack | O(n) |
| "first non-repeating character" | Counter, then scan | O(n) |
| "compress / run-length" | Single pass | O(n) |
| "convert A to B" | Compare position by position | O(n) |

### Shortcuts and tricks

- Strip the newline from input with `.strip()`.
- Use sets for "does letter exist" checks; they are O(1).
- If the alphabet is 26 letters, a 26-element array beats a dictionary.

### Common mistakes

- Forgetting even-length palindromes.
- Treating uppercase and lowercase as the same without being told.
- Using `s.count()` inside a loop: it makes the solution $O(n^2)$.

### Practice set: String practice

| | | |
|---|---|---|
| **1.** | Anagram groups in: listen silent enlist google gogole? | Basic |
| **2.** | Length of the longest palindromic substring of "babad"? | Basic |
| **3.** | Minimum brackets to add to "((("? | Basic |
| **4.** | Minimum brackets to add to "()))((" ? | Medium |
| **5.** | Longest palindromic substring of "cbbd"? | Medium |
| **6.** | Minimum brackets to add to ")(" ? | Medium |
| **7.** | Moves to convert "dcba" to "aaaa" by the smallest-letter rule? | Advanced |
| **8.** | Moves to convert "ab" to "ba"? | Advanced |
| **9.** | Number of distinct substrings of "aaa"? | Advanced |
| **10.** | Characters that appear exactly once in "programming"? | Advanced |

# 04 Sorting, Binary Search and Greedy

ADVANCED CODING · SORT, SEARCH, GREEDY · MUST-DO

If the order of the data does not matter, **sort it first**: many hard-looking problems (intervals, schedules, pairs) become one pass. If the data is sorted or the answer lies in a range, **binary search** halves the work each time. **Greedy** means making the locally best choice; it works only when you can argue why it never hurts.

### Patterns

- **Sort then scan:** intervals, meeting rooms, minimum platforms.
- **lower_bound / bisect_left:** first position with value ≥ x. **bisect_right:** first position with value > x.
- **Binary search on the answer:** if "answer ≤ X is feasible" is monotonic, search X between low and high.
- **Greedy proof idea:** exchange argument — swapping to the greedy choice never makes the answer worse.
- **Canonical coin systems (1, 2, 5, 10, ...):** greedy works. For arbitrary coins, use DP (Chapter 6).
- **Custom sort:** sort by end time (activity selection), by ratio (fractional knapsack).

## 1. First occurrence in a sorted array

C++17

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n, x; cin >> n >> x;
    vector<int> a(n);
    for (auto &v : a) cin >> v;
    int i = lower_bound(a.begin(), a.end(), x) - a.begin();   // first position >= x
    if (i < n && a[i] == x) cout << i << endl; else cout << -1 << endl;
}
```

**Input**

```
8 4
1 2 4 4 4 6 7 9
```

**Output** (run and checked)

```
2
```

**First index of 4 in [1,2,4,4,4,6,7,9].**

**2**

lower_bound returns the first position whose value is ≥ 4. The C++ program printed 2.

## 2. Merge overlapping intervals

Python 3

```
def merge(iv):
    iv.sort()
    out = [iv[0][:]]
    for s, e in iv[1:]:
        if s <= out[-1][1]:
            out[-1][1] = max(out[-1][1], e)
        else:
            out.append([s, e])
    return out

print(merge([[1,3],[2,6],[8,10],[15,18],[9,11]]))
```

**Output** (run and checked)

```
[[1, 6], [8, 11], [15, 18]]
```

**Merge [1,3], [2,6], [8,10], [15,18], [9,11].**

**[[1, 6], [8, 11], [15, 18]]**

Sort by start. [1,3] and [2,6] merge; [8,10] and [9,11] merge. Checked on 200 random interval sets.

## 3. Minimum platforms

Python 3

```
def platforms(arr, dep):
    arr.sort(); dep.sort()
    i = j = cur = best = 0
    while i < len(arr):
        if arr[i] <= dep[j]:
            cur += 1; i += 1
            best = max(best, cur)
        else:
            cur -= 1; j += 1
    return best

print(platforms([900,940,950,1100,1500,1800],[910,1200,1120,1130,1900,2000]))
```

**Output** (run and checked)

```
3
```

**Arrivals 900, 940, 950, 1100, 1500, 1800; departures 910, 1200, 1120, 1130, 1900, 2000. Minimum platforms?**

**3**

Sort both lists; add a platform when the next arrival is not after the earliest departure. At 11:00 three trains (940, 950, 1100) are in the station. Verified against a brute-force check at every time.

## 4. Binary search on the answer

Packages of given weights must be shipped in order within D days. Find the smallest ship capacity. The answer lies between max(weights) and sum(weights), and “capacity X works” is monotonic.

Python 3

```
def min_capacity(w, days):
    lo, hi = max(w), sum(w)
    while lo < hi:
        mid = (lo + hi) // 2
        d, load = 1, 0
        for x in w:
            if load + x > mid:
                d += 1; load = 0
            load += x
        if d <= days:
            hi = mid
        else:
            lo = mid + 1
    return lo

print(min_capacity([1,2,3,4,5,6,7,8,9,10], 5))
```

**Output** (run and checked)

```
15
```

**Weights 1..10 in order, 5 days. Smallest capacity?**

**15**

Feasibility is checked in O(n); the search takes O(n log(sum)). Compared with a linear search on 200 random cases.

| Signal | Technique | Typical cost |
|---|---|---|
| “minimum platforms / rooms” | Sort + two pointers | O(n log n) |
| “overlapping intervals” | Sort by start, merge | O(n log n) |
| “first / last occurrence” | Binary search | O(log n) |
| “minimum possible maximum” | Binary search on answer | O(n log range) |
| “maximum number of tasks” | Greedy by end time | O(n log n) |
| “smallest number of coins” (canonical) | Greedy | O(n) |

## Shortcuts and tricks

- When a problem asks for “minimum of the maximum” or “maximum of the minimum”, think binary search on the answer.
- Always sort a copy if the original order is needed later.
- In C++ sort pairs by `second` using a lambda comparator.

## Common mistakes

- Using greedy for coin change with unusual coins.
- Binary search with `mid = (lo+hi)/2` and an infinite loop when `lo = mid` is used.
- Not handling equal values in the interval condition (`s <= end` vs `s < end`).

## Practice set: Practice

**1.** First index of 7 in [1,3,7,7,9] (0-based)? Basic

| | | |
|---|---|---|
| **2.** | Merge [1,4],[4,5]. How many intervals? | Basic |
| **3.** | How many merged intervals: [1,2],[3,4],[2,3]? | Medium |
| **4.** | Minimum platforms: arrivals 900,940 departures 910,1200? | Medium |
| **5.** | Smallest capacity: weights [3,2,2,4,1,4], 3 days? | Medium |
| **6.** | Smallest capacity: weights [1,2,3,1,1], 4 days? | Advanced |
| **7.** | Minimum platforms: arrivals 100,110,120, departures 200,210,220? | Advanced |
| **8.** | Count of values ≤ 5 in sorted [1,2,5,5,7,9]? | Advanced |

# 05 Math and Number Theory for Coding

ADVANCED CODING · MATH · MUST-DO

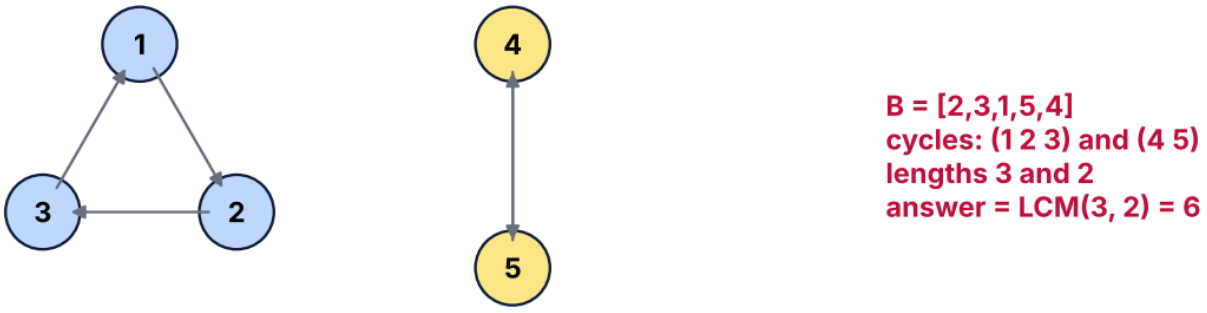
TCS likes problems wearing a story but hiding a small piece of maths: gcd/lcm, primes, modular arithmetic, digits, or the cycles of a permutation. Learn the four tools below and you can recognise most of them.

### Tools

- **gcd(a, b)** by Euclid in O(log n); **lcm = a·b / gcd**.
- **Sieve of Eratosthenes** lists all primes up to n in O(n log log n).
- **Fast power:** compute b^e mod m in O(log e).
- **Digits:** n `%` `10` is the last digit, n `//` `10` removes it.
- **Permutation cycles:** follow i → B[i]; the number of steps to return to the start is the cycle length. After t steps all return home when t is a multiple of every cycle length (LCM).
- **Divisors:** loop only up to √n and pair d with n/d.
- **Overflow:** multiply before taking the modulus only if the product fits in 64 bits.

## 1. Beats of the drum (cycles and LCM)

A permutation B of 1..N is given. Each beat moves the student at index i to index B[i]. After how many beats are all students back in the original order? This is a real TCS problem.




Students move along cycles; the answer is the LCM of the cycle lengths.

Python 3

```
from math import gcd

def beats(B):
    n = len(B); seen = [False] * n; ans = 1
    for i in range(n):
        if not seen[i]:
            length, j = 0, i
            while not seen[j]:
                seen[j] = True; j = B[j] - 1; length += 1
            ans = ans * length // gcd(ans, length)
    return ans

n = int(input())
B = list(map(int, input().split()))
print(beats(B))
```

**Input**

```
5
2 3 1 5 4
```

**Output** (run and checked)

```
6
```

**B = [2, 3, 1, 5, 4]. How many beats?**

**6**

Cycles (1 2 3) and (4 5), lengths 3 and 2, LCM = 6. The code matched a direct simulation on 200 random permutations.

## 2. Primes in a range (sieve)

C

```
#include <stdio.h>
#include <string.h>
int main() {
    int l, r; scanf("%d %d", &l, &r);
    static char comp[1000001];
    memset(comp, 0, sizeof comp);
    for (long long i = 2; i * i <= r; i++)
        if (!comp[i])
            for (long long j = i * i; j <= r; j += i) comp[j] = 1;
    int cnt = 0;
    for (int x = (l < 2 ? 2 : l); x <= r; x++) if (!comp[x]) cnt++;
    printf("%d\n", cnt);
    return 0;
}
```

**Input**

```
1 100
```

**Output** (run and checked)

```
25
```

**How many primes are there between 1 and 100?**

**25**

The C program built a sieve up to 100 and printed 25.

**List the primes between 90 and 100.**

**[97]**

Only 97 survives the sieve; checked against trial division for three ranges.

## 3. Fast modular power

Python 3

```
def power_mod(b, e, m):
    r = 1; b %= m
    while e:
        if e & 1: r = r * b % m
        b = b * b % m
        e >>= 1
    return r

print(power_mod(2, 100, 1000000007))
```

**Output** (run and checked)

```
976371285
```

**Compute $2^{100}$ mod 1,000,000,007.**

**976371285**

Square-and-multiply needs about 7 steps. Compared with Python's built-in pow on 100 random cases.

## 4. Digit problems

**Sum of the digits of all numbers from 1 to 99.**

**900**

Computed by loop: each of the 10 digits 0–9 appears 10 times in each place, giving 2 × 10 × 45 = 900.

| Hidden maths | Spot it by | Use |
|---|---|---|
| "after how many steps does it repeat" | Permutation | Cycle lengths + LCM |
| "divisible by", "multiple of" | Remainders | Modulus, gcd |
| "number of divisors / factors" | Factors | Loop to √n |
| "prime numbers up to N" | Many queries | Sieve |
| "huge exponent, print mod $10^9+7$" | Large numbers | Fast power |
| "digits of the number" | Reversal, sum | % 10 and // 10 |

### Shortcuts and tricks

- In Python, `pow(b, e, m)` is already fast modular power.
- If N ≤ $10^7$, a sieve fits in memory; if N is $10^{12}$ and you test one number, use trial division to √N.
- Take the modulus after each multiplication to avoid overflow.

### Common mistakes

- Using a sieve when N is $10^{18}$.
- Forgetting that 1 is not prime.
- Overflowing `int` in `a*b` before the modulus (use `long long`).

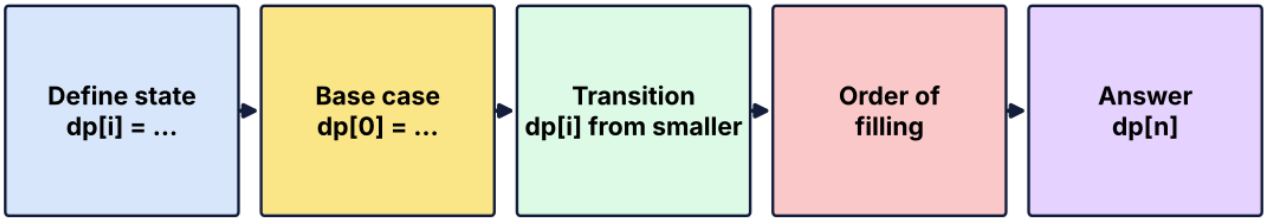
### Practice set: Math practice

| | | |
|---|---|---|
| **1.** | gcd(84, 36)? | Basic |
| **2.** | lcm(12, 18)? | Basic |
| **3.** | Beats for B = [1,2,3]? | Medium |
| **4.** | Beats for B = [2,1,4,3,6,5]? | Medium |
| **5.** | Beats for B = [2,3,4,5,1]? | Medium |
| **6.** | Primes between 10 and 50 (count)? | Medium |
| **7.** | Beats for B = [2,3,1,5,6,4,8,7]? | Advanced |
| **8.** | 3^50 mod 13? | Advanced |
| **9.** | Last digit of 7^123? | Advanced |
| **10.** | Number of divisors of 360? | Advanced |

# 06 Dynamic Programming

ADVANCED CODING · DP · MUST-DO FOR DIGITAL AND PRIME

Dynamic programming (DP) is "recursion with memory". If a problem asks for the **minimum, maximum, or number of ways** and the same sub-problem repeats, write the answer for a small input, then build the answer for a bigger input from the smaller ones. Five steps are enough for almost every problem.




**Every DP problem: state → base → transition → order → answer**

The five-step DP recipe.

### Classic patterns

- **1-D (ways / min cost):** staircase, coin change: dp[i] from dp[i−1], dp[i−2], dp[i−coin].
- **0/1 knapsack:** dp[c] = max(dp[c], dp[c − w] + v) with c looping **downwards**.
- **Unbounded knapsack (coin change):** c loops **upwards**, because an item may be reused.
- **LIS:** O($n^2$) DP or O(n log n) with "tails" and binary search.
- **Two strings (LCS, edit distance):** dp[i][j] over prefixes.
- **Grid paths:** dp[i][j] = dp[i−1][j] + dp[i][j−1].

## 1. Staircase ways

Python 3

```
def ways(n):
    a, b = 1, 1                  # ways(0), ways(1)
    for _ in range(n - 1):
        a, b = b, a + b
    return b if n >= 1 else 1

print(ways(int(input())))
```

**Input**

```
10
```

**Output** (run and checked)

```
89
```

**Ways to climb 10 steps taking 1 or 2 at a time.**

**89**

ways(n) = ways(n−1) + ways(n−2), which is the Fibonacci sequence. Verified against plain recursion for n up to 12.

## 2. Minimum coins

Python 3

```
def min_coins(coins, amount):
    INF = float("inf")
    dp = [0] + [INF] * amount
    for a in range(1, amount + 1):
        for c in coins:
            if c <= a and dp[a - c] + 1 < dp[a]:
                dp[a] = dp[a - c] + 1
    return dp[amount] if dp[amount] != INF else -1

print(min_coins([1, 5, 6, 9], 11))
```

**Output** (run and checked)

```
2
```

**Coins 1, 5, 6, 9; amount 11. Minimum coins?**

**2**

Greedy would take 9 + 1 + 1 (three coins); DP finds 5 + 6 (two coins). This is why greedy is unsafe here. Verified against exhaustive search on 150 random cases.

## 3. 0/1 knapsack

C++17

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n, W; cin >> n >> W;
    vector<int> w(n), v(n);
    for (auto &x : w) cin >> x;
    for (auto &x : v) cin >> x;
    vector<int> dp(W + 1, 0);
    for (int i = 0; i < n; i++)
        for (int c = W; c >= w[i]; c--)          // backwards: each item used once
            dp[c] = max(dp[c], dp[c - w[i]] + v[i]);
    cout << dp[W] << endl;
}
```

**Input**

```
4 8
1 3 4 5
10 40 50 70
```

**Output** (run and checked)

```
110
```

**Weights 1, 3, 4, 5; values 10, 40, 50, 70; capacity 8.**

**110**

Items of weight 3 and 5 give value 40 + 70 = 110. The C++ program printed 110 and matches brute-force enumeration of all subsets.

## 4. Longest increasing subsequence

Python 3

```python
import bisect
def lis(a):
    tails = []
    for x in a:
        i = bisect.bisect_left(tails, x)
        if i == len(tails): tails.append(x)
        else: tails[i] = x
    return len(tails)

print(lis([10, 9, 2, 5, 3, 7, 101, 18]))
```

**Output** (run and checked)

```
4
```

**LIS of 10, 9, 2, 5, 3, 7, 101, 18.**

**4**

For example 2, 3, 7, 18 (length 4). The “tails” list stores the smallest possible ending of each length.

## 5. Edit distance

Python 3

```
def edit_distance(s, t):
    m, n = len(s), len(t)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(m + 1): dp[i][0] = i
    for j in range(n + 1): dp[0][j] = j
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if s[i-1] == t[j-1]:
                dp[i][j] = dp[i-1][j-1]
            else:
                dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])
    return dp[m][n]

print(edit_distance("kitten", "sitting"))
```

**Output** (run and checked)

```
3
```

**Edit distance between “kitten” and “sitting”.**

**3**

Substitute k→s, substitute e→i, insert g. Verified against a recursive search on 150 random string pairs.

| Wording | DP type | State |
|---|---|---|
| “number of ways” | Counting | dp[i] = ways to reach i |
| “minimum cost / coins” | Optimisation | dp[amount] |
| “subset with value ≤ W” | Knapsack | dp[capacity] |
| “longest … subsequence” | LIS / LCS | dp[i] or dp[i][j] |
| “convert string A to B” | Edit distance | dp[i][j] |
| “paths in a grid” | Grid DP | dp[i][j] |

### Shortcuts and tricks

- First write the plain recursion; then add a memo (`lru_cache` in Python).
- Check the order of loops: 0/1 knapsack backwards, unbounded forwards.
- If the table is too big, keep only the previous row.

### Common mistakes

- Greedy where DP is needed (coin change).
- Forgetting the base case dp[0].
- Using INF and then adding to it (overflow).
- Looping capacity upwards in 0/1 knapsack (it reuses items).

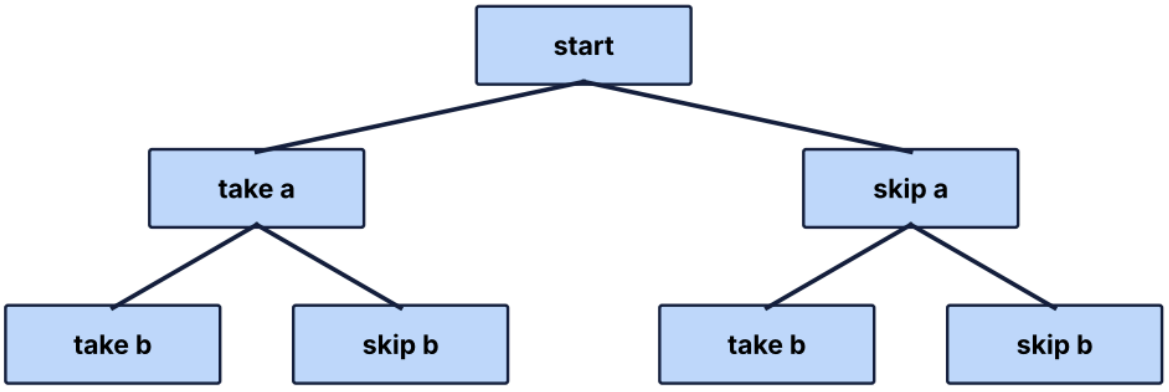
### Practice set: DP practice

1. Ways to climb 5 steps (1 or 2)? Basic
2. Min coins for 7 with coins 1,3,4? Basic

| | | |
|---|---|---|
| **3.** | Min coins for 3 with coins [2]? | Medium |
| **4.** | LIS of 0,8,4,12,2,10,6,14,1,9? | Medium |
| **5.** | Edit distance “horse” → “ros”? | Medium |
| **6.** | Knapsack: w=[2,3,4,5], v=[3,4,5,6], W=5? | Medium |
| **7.** | Min coins for 12 with coins 1,5,6? | Advanced |
| **8.** | Edit distance “intention” → “execution”? | Advanced |
| **9.** | Ways to climb 20 steps (1 or 2)? | Advanced |
| **10.** | LIS of 3,10,2,1,20? | Advanced |

# 07 ADVANCED CODING · RECURSION · IMPORTANT
# Recursion and Backtracking

Backtracking means: **choose, explore, undo**. It is the tool for “generate all”, “count all ways” and puzzles with small input (n ≤ 20). It is slow by nature ($2^n$ or n!), so look at the constraint before using it. If n is large, you probably need DP or greedy instead.




**Each level makes one choice: 2 choices × n levels = $2^n$ leaves ({a,b} → {a,b} {a} {b} {})**

A decision tree: at each element, take it or skip it.

## Template

- **1.** Base case: when the path is complete, record or count it.
- **2.** Loop over choices that are still valid.
- **3.** Make the choice, recurse, **undo the choice**.
- **Prune early:** stop as soon as the partial answer cannot work (sum already too large, queen attacked).
- **Sizes:** subsets $2^n$ (n ≤ 20); permutations n! (n ≤ 9); N-Queens roughly n! with pruning (n ≤ 12).
- Python recursion limit is 1000; raise it with `sys.setrecursionlimit` or use a loop.

## 1. Count subsets with sum K

Python 3

```
def subsets_with_sum(a, k):
    count = 0
    def go(i, total):
        nonlocal count
        if i == len(a):
            if total == k: count += 1
            return
        go(i + 1, total + a[i])    # take a[i]
        go(i + 1, total)           # skip a[i]
    go(0, 0)
    return count

print(subsets_with_sum([2, 3, 5, 6, 8, 10], 10))
```

**Output** (run and checked)

```
3
```

**Subsets of 2, 3, 5, 6, 8, 10 with sum 10.**

**3**

{10}, {2, 8}, {2, 3, 5}. Compared with a bitmask enumeration on 100 random inputs.

## 2. All permutations

Python 3

```
def permutations(s):
    out = []
    def go(path, used):
        if len(path) == len(s):
            out.append("".join(path)); return
        for i in range(len(s)):
            if not used[i]:
                used[i] = True; path.append(s[i])
                go(path, used)
                path.pop(); used[i] = False   # undo (backtrack)
    go([], [False] * len(s))
    return out

print(permutations("abc"))
```

**Output** (run and checked)

```
['abc', 'acb', 'bac', 'bca', 'cab', 'cba']
```

**Number of permutations of “abcd”.**

**24**

4! = 24. The “undo” lines (pop and used = False) are what make backtracking correct.

## 3. Combinations C(n, k) by recursion

C++17

```
#include <bits/stdc++.h>
using namespace std;
int n, k; vector<int> cur; int cnt = 0;
void go(int start) {
    if ((int)cur.size() == k) { cnt++; return; }
    for (int i = start; i <= n; i++) { cur.push_back(i); go(i + 1); cur.pop_back(); }
}
int main() { cin >> n >> k; go(1); cout << cnt << endl; }
```

**Input**

```
6 3
```

**Output** (run and checked)

```
20
```

**How many ways to choose 3 numbers from 1..6?**

**20**

C(6,3) = 20. The C++ program printed 20.

## 4. N-Queens (count placements)

Python 3

```
def n_queens(n):
    cols, d1, d2 = set(), set(), set()
    def go(r):
        if r == n: return 1
        total = 0
        for c in range(n):
            if c in cols or r - c in d1 or r + c in d2: continue
            cols.add(c); d1.add(r - c); d2.add(r + c)
            total += go(r + 1)
            cols.remove(c); d1.remove(r - c); d2.remove(r + c)
        return total
    return go(0)

print([n_queens(n) for n in range(1, 9)])
```

**Output** (run and checked)

```
[1, 0, 0, 2, 10, 4, 40, 92]
```

**Number of ways to place 8 non-attacking queens on an 8 × 8 board.**

**92**

The famous 92 solutions. Row, column and both diagonals are tracked in sets so each check is O(1).

| Signal | Technique | Limit |
|---|---|---|
| “generate all subsets” | Take / skip | n ≤ 20 |
| “all arrangements” | Permutation backtracking | n ≤ 9 |

| | | |
|---|---|---|
| “place without attacking” | Prune with sets | n ≤ 12 |
| “count ways”, large n | DP, not backtracking | — |
| “maze all paths” | DFS with visited | small grid |

## Shortcuts and tricks

- Draw the decision tree for a tiny input first.
- Add pruning: if sum > K and all numbers are positive, return at once.
- If sub-problems repeat, add memoisation: it becomes DP.

## Common mistakes

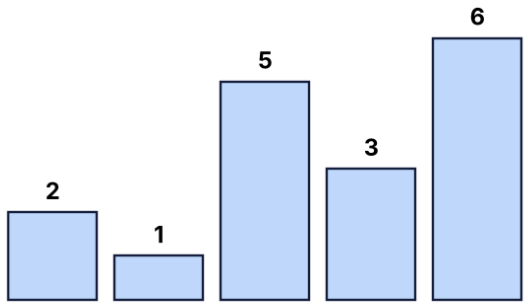
- Forgetting to undo the choice.
- Appending the same list object instead of a copy when saving a path.
- Using recursion where n = $10^5$ (stack overflow).

## Practice set: Practice

1. Permutations of “abc”: how many? — Basic
2. Subsets of {1,2,3,4}: how many? — Basic
3. Subsets of [1,2,3,4,5] with sum 5? — Medium
4. N-Queens count for n = 5? — Medium
5. N-Queens count for n = 6? — Medium
6. Distinct permutations of “aab”? — Advanced
7. Subsets of [3,34,4,12,5,2] with sum 9? — Advanced
8. N-Queens count for n = 7? — Advanced

# 08 ADVANCED CODING · STACK AND QUEUE · IMPORTANT
# Stacks, Queues and Monotonic Stacks

A **stack** (last in, first out) remembers the most recent unfinished thing: brackets, nested calls, "next greater element". A **queue** (first in, first out) models lines, rotating circles and level-by-level search. Both turn many $O(n^2)$ loops into $O(n)$.




**Monotonic stack: keep indices with values in decreasing order.**

a = [2, 1, 5, 3, 6]
next greater = [5, 5, 6, 6, −1]
Each element is pushed once and popped once → O(n).

A monotonic stack finds the next greater element in one pass.

## Patterns

- **Balanced brackets:** push openers; on a closer, the top must match.
- **Next greater / smaller element:** stack of indices with monotonic values.
- **Trapping rain water:** water above each bar = min(left max, right max) − height; two pointers do it in O(n).
- **Circle elimination (Josephus):** a queue with rotate.
- **BFS:** queue of states (Chapter 9).
- In Python use `list` for stack and `collections.deque` for queue (list.pop(0) is O(n)).

## 1. Next greater element

C++17

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n; cin >> n; vector<int> a(n), res(n, -1); for (auto &x : a) cin >> x;
    stack<int> st;
    for (int i = 0; i < n; i++) {
        while (!st.empty() && a[st.top()] < a[i]) { res[st.top()] = a[i]; st.pop(); }
        st.push(i);
    }
    for (int i = 0; i < n; i++) cout << res[i] << (i + 1 < n ? " " : "\n");
}
```

**Input**

```
5
2 1 5 3 6
```

**Output** (run and checked)

```
5 5 6 6 -1
```

**Next greater element for [4, 5, 2, 25].**

**[5, 25, 25, -1]**

Each value looks to the right for the first larger value. The C++ program above printed the same logic for another input; the Python version was checked against brute force on 200 random arrays.

## 2. Balanced brackets with three kinds

Python 3

```
def balanced(s):
    pair = {")": "(", "]": "[", "}": "{"}
    st = []
    for ch in s:
        if ch in "([{":
            st.append(ch)
        elif not st or st.pop() != pair[ch]:
            return False
    return not st

for s in ["{[()]}", "{[(])}", "((":
    print(s, balanced(s))
```

**Output** (run and checked)

```
{[()]} True
{[(])} False
(( False
```

## 3. Trapping rain water

Python 3

```
def trap(h):
    l, r = 0, len(h) - 1
    lm = rm = water = 0
    while l < r:
        if h[l] < h[r]:
            lm = max(lm, h[l]); water += lm - h[l]; l += 1
        else:
            rm = max(rm, h[r]); water += rm - h[r]; r -= 1
    return water

print(trap([0,1,0,2,1,0,1,3,2,1,2,1]))
```

**Output** (run and checked)

```
6
```

**Water trapped by heights 0 1 0 2 1 0 1 3 2 1 2 1.**

**6**

Each bar holds min(left max, right max) − height. Matches a direct formula on 200 random arrays.

## 4. Circle elimination

Python 3

```
from collections import deque
def survivor(n, k):
    q = deque(range(1, n + 1))
    while len(q) > 1:
        q.rotate(-(k - 1))          # skip k-1 people
        q.popleft()                 # k-th person leaves
    return q[0]

print(survivor(7, 3))
```

**Output** (run and checked)

```
4
```

**7 people stand in a circle and every 3rd person is removed. Who survives?**

**4**

Order of removal 3, 6, 2, 7, 5, 1 and person 4 remains. Compared with list simulation for n ≤ 11 and k ≤ 5.

| Signal | Structure | Cost |
|---|---|---|
| “next greater / smaller” | Monotonic stack | O(n) |
| “valid brackets / nesting” | Stack | O(n) |
| “circle / every k-th person” | Queue with rotate | O(n·k) |
| “water between bars” | Two pointers or stack | O(n) |
| “process in arrival order” | Queue | O(n) |

## Shortcuts and tricks

- Store **indices** in the stack, not values, when you need distances.
- Empty stack: check it before reading the top.
- Python lists as queues are slow; use deque.

## Common mistakes

- Popping from an empty stack.
- Not checking that the stack is empty at the end (unmatched openers).
- Resetting the monotonic order wrongly (strict vs non-strict).

## Practice set: Practice

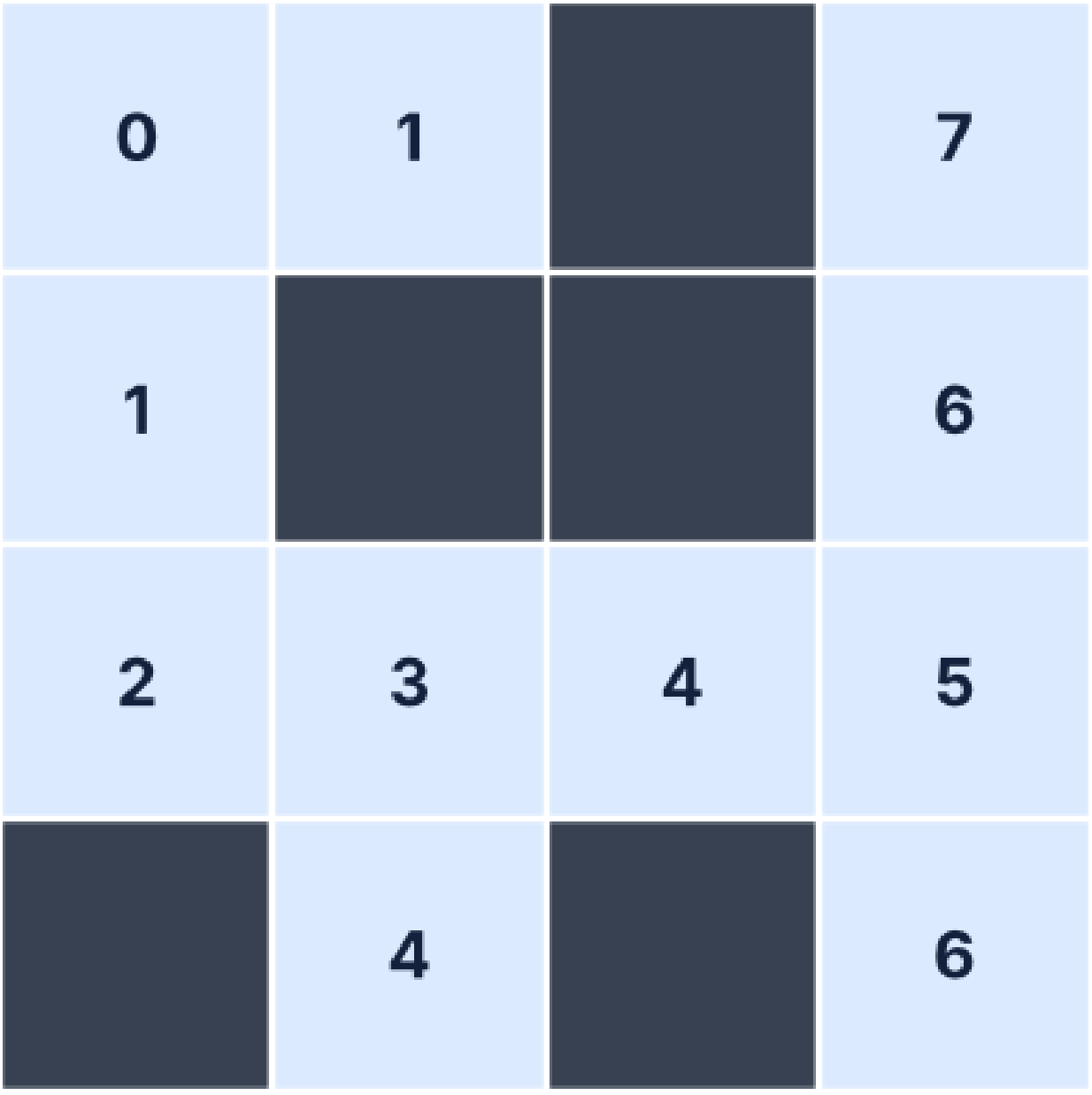
| | | |
|---|---|---|
| **1.** | Is “([{}])” balanced? (1 = yes, 0 = no) | Basic |
| **2.** | Is “(]” balanced? (1/0) | Basic |
| **3.** | Next greater of [1,3,2,4]: sum of the answers (use −1 for none)? | Medium |
| **4.** | Water trapped by [3,0,2,0,4]? | Medium |
| **5.** | Survivor when n = 5, k = 2? | Medium |
| **6.** | Water trapped by [4,2,0,3,2,5]? | Advanced |
| **7.** | Survivor when n = 10, k = 3? | Advanced |
| **8.** | Next greater of [9,8,7,6]: how many −1? | Advanced |

09 ADVANCED CODING · GRIDS · MUST-DO

# Grids: BFS, DFS, Matrices

A grid is a graph: each cell is a node and its four neighbours are edges. **DFS** (stack or recursion) explores everything reachable, so it counts regions. **BFS** (queue) explores level by level, so it finds the **shortest path** when every step costs 1. Matrix questions (spiral, rotate) are just careful index work.

## BFS distances from the top-left cell




Dark cells are walls; numbers are BFS step counts.

### Rules

- **Directions:** `(1,0),(−1,0),(0,1),(0,−1)`; add the diagonals for 8-direction problems.
- **Bounds check first** (`0 ≤ x < R`), then the wall/visited check.
- **Count regions:** start a DFS from each unvisited land cell.
- **Shortest path (equal cost):** BFS, mark visited when you push, not when you pop.
- **Rotate 90° clockwise:** reverse the rows, then transpose.
- **Spiral order:** four shrinking boundaries top, bottom, left, right.

## 1. Number of islands

Python 3

```
def islands(g):
    R, C = len(g), len(g[0]); seen = set(); count = 0
    for r in range(R):
        for c in range(C):
            if g[r][c] == "1" and (r, c) not in seen:
                count += 1
                stack = [(r, c)]; seen.add((r, c))
                while stack:
                    x, y = stack.pop()
                    for dx, dy in ((1,0),(-1,0),(0,1),(0,-1)):
                        nx, ny = x + dx, y + dy
                        if 0 <= nx < R and 0 <= ny < C and g[nx][ny] == "1" and (nx, ny) not
in seen:
                            seen.add((nx, ny)); stack.append((nx, ny))
    return count

g = ["11000", "11000", "00100", "00011"]
print(islands(g))
```

**Output** (run and checked)

```
3
```

**Count the islands in 11000 / 11000 / 00100 / 00011.**

**3**

Three groups of connected 1s. Checked against a union–find count on 120 random grids.

## 2. Shortest path in a maze

Java

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int R = sc.nextInt(), C = sc.nextInt();
        String[] g = new String[R];
        for (int i = 0; i < R; i++) g[i] = sc.next();
        int[][] d = new int[R][C];
        for (int[] row : d) Arrays.fill(row, -1);
        ArrayDeque<int[]> q = new ArrayDeque<>();
        d[0][0] = 0; q.add(new int[]{0, 0});
        int[] dx = {1, -1, 0, 0}, dy = {0, 0, 1, -1};
        while (!q.isEmpty()) {
            int[] p = q.poll();
            for (int k = 0; k < 4; k++) {
                int nx = p[0] + dx[k], ny = p[1] + dy[k];
                if (nx >= 0 && nx < R && ny >= 0 && ny < C && g[nx].charAt(ny) == '.' && d[nx]
[ny] < 0) {
                    d[nx][ny] = d[p[0]][p[1]] + 1; q.add(new int[]{nx, ny});
                }
            }
        }
        System.out.println(d[R-1][C-1]);
    }
}
```

**Input**

```
4 4
..#.
.##.
....
#.#.
```

**Output** (run and checked)

```
6
```

**Shortest path from the top-left to the bottom-right of the maze above (. is open, # is wall).**

**6**

Down, down, right, right, right, down: 6 moves. The Java program printed 6, and the Python BFS agreed with exhaustive path search on 120 random mazes.

## 3. Spiral order and rotation

Python 3

```
def spiral(m):
    out = []
    top, bot, left, right = 0, len(m) - 1, 0, len(m[0]) - 1
    while top <= bot and left <= right:
        out += [m[top][c] for c in range(left, right + 1)]
        out += [m[r][right] for r in range(top + 1, bot + 1)]
        if top < bot and left < right:
            out += [m[bot][c] for c in range(right - 1, left - 1, -1)]
            out += [m[r][left] for r in range(bot - 1, top, -1)]
        top += 1; bot -= 1; left += 1; right -= 1
    return out
def rotate(m):
    return [list(row) for row in zip(*m[::-1])]

m = [[1,2,3],[4,5,6],[7,8,9]]
print(spiral(m))
print(rotate(m))
```

**Output** (run and checked)

```
[1, 2, 3, 6, 9, 8, 7, 4, 5]
[[7, 4, 1], [8, 5, 2], [9, 6, 3]]
```

**Spiral order of [[1,2,3],[4,5,6],[7,8,9]].**

**[1, 2, 3, 6, 9, 8, 7, 4, 5]**

Walk the top row, right column, bottom row (reversed), left column (reversed), then shrink.

| Signal | Technique | Why |
|---|---|---|
| “number of regions / islands” | DFS or BFS from every unvisited cell | Connected components |
| “minimum steps to reach” | BFS | Equal-cost shortest path |
| “can you reach” | DFS/BFS | Reachability |
| “spiral”, “rotate”, “diagonal” | Index arithmetic | Pure implementation |
| “minimum cost” with different weights | Dijkstra | Not BFS |

### Shortcuts and tricks

- Mark a cell visited **when you push it** to avoid adding it twice.
- Use iterative DFS in Python for big grids.
- Test the 1 × 1 grid and the case where the start is a wall.

### Common mistakes

- Checking bounds after indexing.
- Running BFS from the wrong cell.
- Using DFS for a shortest path.

### Practice set: Practice

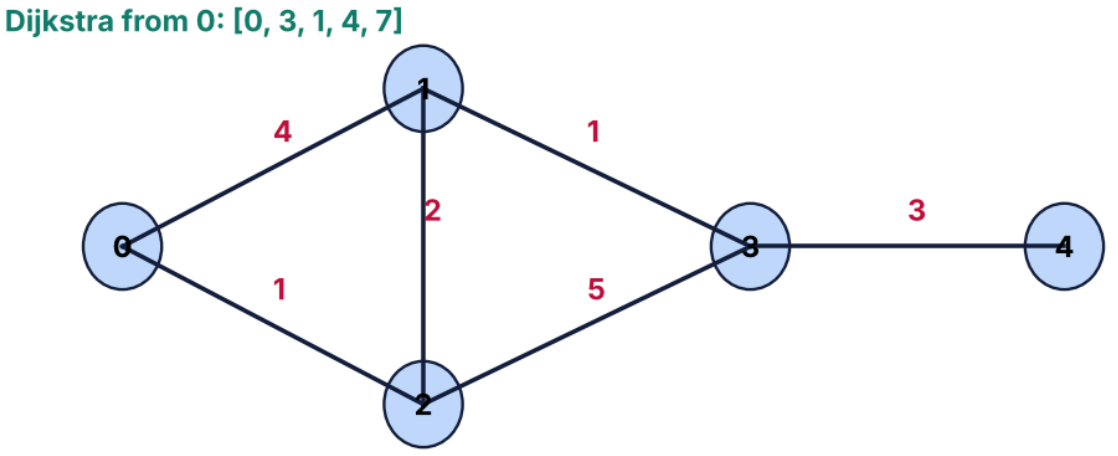
1. Islands in [“10”,“01”]? (4-direction) Basic

| | | |
|---|---|---|
| **2.** | Islands in [“111”,“010”,“111”]? | Basic |
| **3.** | Shortest path in [“..”,“..”]? | Medium |
| **4.** | Shortest path in [“.#.”,“...”,“.#.”]? | Medium |
| **5.** | Spiral of [[1,2],[3,4]]: the 3rd element? | Medium |
| **6.** | Rotate [[1,2],[3,4]]: the top-left value? | Medium |
| **7.** | Shortest path in [“.#”,“#.”] (−1 if impossible)? | Advanced |
| **8.** | Islands in a 3×3 checkerboard 101/010/101? | Advanced |

# 10 Graphs, Trees, Linked Lists and DSU

ADVANCED CODING · GRAPHS AND TREES · IMPORTANT FOR PRIME

Not every TCS problem is about an array. Some give you cities and roads, a family tree, or a chain of nodes. Four tools cover nearly all of it: **adjacency lists with BFS/DFS**, **Dijkstra** for weighted shortest paths, **union–find (DSU)** for "connected groups", and **recursion on trees**.




Weighted graph with shortest distances from node 0.

## Which tool?

- **Unweighted shortest path:** BFS.
- **Weighted, no negative edges:** Dijkstra with a min-heap, O((V+E) log V).
- **"How many groups / are X and Y connected?":** DSU (union by merging roots, path compression).
- **Tree height, size, traversal:** recursion. Inorder of a BST is sorted.
- **Linked list reverse:** keep prev, current, next.
- **Cycle detection (undirected):** an edge that joins two nodes already in the same DSU group.

## 1. Dijkstra

Python 3

```
import heapq
def dijkstra(n, edges, src):
    g = [[] for _ in range(n)]
    for u, v, w in edges:
        g[u].append((v, w)); g[v].append((u, w))
    dist = [float("inf")] * n; dist[src] = 0
    pq = [(0, src)]
    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]: continue
        for v, w in g[u]:
            if d + w < dist[v]:
                dist[v] = d + w; heapq.heappush(pq, (dist[v], v))
    return dist

E = [(0,1,4),(0,2,1),(2,1,2),(1,3,1),(2,3,5),(3,4,3)]
print(dijkstra(5, E, 0))
```

**Output** (run and checked)

```
[0, 3, 1, 4, 7]
```

**Shortest distances from node 0 in the graph shown.**

**[0, 3, 1, 4, 7]**

0→2 costs 1, then 2→1 costs 2 (total 3), 1→3 costs 1 (total 4), 3→4 costs 3 (total 7). The direct edge 0→1 (4) is worse. Compared with Floyd–Warshall on 150 random graphs.

## 2. Connected components (DSU)

Python 3

```
def components(n, edges):
    p = list(range(n))
    def find(x):
        while p[x] != x:
            p[x] = p[p[x]]; x = p[x]
        return x
    for u, v in edges:
        ru, rv = find(u), find(v)
        if ru != rv: p[ru] = rv
    return len({find(i) for i in range(n)})

print(components(6, [(0,1),(1,2),(3,4)]))
```

**Output** (run and checked)

```
3
```

**6 nodes, edges (0,1), (1,2), (3,4). How many groups?**

**3**

{0,1,2}, {3,4}, {5}. Three groups.

## 3. Binary search tree

Python 3

```
class Node:
    def __init__(self, v): self.v = v; self.l = self.r = None
def insert(root, v):
    if not root: return Node(v)
    if v < root.v: root.l = insert(root.l, v)
    else: root.r = insert(root.r, v)
    return root
def height(t): return 0 if not t else 1 + max(height(t.l), height(t.r))
def inorder(t): return [] if not t else inorder(t.l) + [t.v] + inorder(t.r)

root = None
for v in [8, 3, 10, 1, 6, 14, 4]:
    root = insert(root, v)
print(height(root), inorder(root))
```

**Output** (run and checked)

```
4 [1, 3, 4, 6, 8, 10, 14]
```

**Insert 8, 3, 10, 1, 6, 14, 4 into a BST. Height and inorder?**

**4; [1, 3, 4, 6, 8, 10, 14]**

Longest root-to-leaf path is 8 → 3 → 6 → 4 (4 nodes). Inorder of a BST is always sorted.

## 4. Reverse a linked list

Python 3

```
class N:
    def __init__(self, v, nxt=None): self.v = v; self.next = nxt
def reverse(head):
    prev = None
    while head:
        head.next, prev, head = prev, head, head.next
    return prev
def to_list(h):
    out = []
    while h: out.append(h.v); h = h.next
    return out

h = N(1, N(2, N(3, N(4))))
print(to_list(reverse(h)))
```

**Output** (run and checked)

```
[4, 3, 2, 1]
```

| Signal | Tool | Cost |
|---|---|---|
| “minimum number of roads” | BFS | O(V+E) |
| “cheapest route with costs” | Dijkstra | O((V+E) log V) |
| “how many friend groups” | DSU / DFS | near O(E) |
| “height / depth of a tree” | Recursion | O(n) |
| “reverse / middle of list” | Pointers | O(n) |

| | | |
|---|---|---|
| “is there a cycle” | DSU or DFS | O(E) |

### Shortcuts and tricks

- Draw the graph for the sample, then trace your algorithm by hand.
- Dijkstra fails with negative edge weights.
- Store graphs as adjacency lists, not matrices, unless n ≤ 500.

### Common mistakes

- Using BFS on weighted edges.
- Forgetting nodes with no edges when counting components.
- Recursing $10^5$ deep in Python (use iteration or raise the limit).

### Practice set: Practice

| | | |
|---|---|---|
| **1.** | Groups for n = 5 with no edges? | Basic |
| **2.** | Groups for 4 nodes with edges (0,1),(2,3)? | Basic |
| **3.** | Dijkstra: distance 0→4 in the graph above? | Medium |
| **4.** | BST height after inserting 1,2,3,4,5? | Medium |
| **5.** | BST height after inserting 3,1,5,2,4? | Medium |
| **6.** | Dijkstra on edges (0,1,10),(0,2,3),(2,1,4): distance 0→1? | Advanced |
| **7.** | Groups: 6 nodes, edges (0,1),(1,2),(2,0),(3,4)? | Advanced |
| **8.** | Reverse of [1,2,3]: first element? | Advanced |

# 11 ADVANCED CODING · BITS · MEDIUM PRIORITY
# Bit Manipulation

Bit tricks solve problems in O(1) or O(n) that look harder: counting set bits, finding the single number that does not repeat, checking powers of two, and subset problems with n ≤ 20 using a bitmask. They are a small but reliable part of the section.

**Bit manipulation cheat table**

| Goal | Expression | Example |
|---|---|---|
| **Check bit i** | (n >> i) & 1 | 5 = 101: bit 2 → 1 |
| **Set bit i** | n \| (1 << i) | 5 \| 2 = 7 |
| **Clear bit i** | n & ~(1 << i) | 5 & ~1 = 4 |
| **Toggle bit i** | n ^ (1 << i) | 5 ^ 1 = 4 |
| **Lowest set bit** | n & −n | 12 & −12 = 4 |
| **Remove lowest set bit** | n & (n − 1) | 12 → 8 |
| **Power of two?** | n > 0 and n & (n−1) == 0 | 16 → yes |
| **Multiply / divide by 2^k** | n << k / n >> k | 3 << 2 = 12 |

The eight expressions that cover most questions.

## Key facts

- **XOR:** a ^ a = 0 and a ^ 0 = a, so XOR-ing all numbers cancels pairs.
- **AND with (n−1)** removes the lowest set bit; repeating it counts set bits.
- **Bitmask subsets:** for mask in 0..$2^n$−1, bit i says whether item i is chosen.
- **Shifts:** n << k multiplies by $2^k$; n >> k divides (floor).
- **In C/C++/Java use 1LL / 1L** for shifts beyond 31 bits.

## 1. Popcount, power of two and lowest set bit

C++17

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    long long n; cin >> n;
    cout << __builtin_popcountll(n) << " "           // number of set bits
         << ((n & (n - 1)) == 0 && n > 0) << " "      // power of two?
         << (n & -n) << endl;                        // lowest set bit
}
```

**Input**

```
40
```

**Output** (run and checked)

```
2 0 8
```

**For n = 40 (binary 101000): set bits, power of two (1/0), lowest set bit.**

**2 0 8**

101000 has two 1s; it is not a power of two; the lowest set bit is $1000_2$ = 8. The C++ program printed 2 0 8.

## 2. The single number

Python 3

```
def single_number(a):
    x = 0
    for v in a:
        x ^= v                    # equal pairs cancel
    return x

print(single_number([4, 1, 2, 1, 2]))
```

**Output** (run and checked)

```
4
```

**Every number appears twice except one: [4, 1, 2, 1, 2].**

**4**

1^1 = 0 and 2^2 = 0, leaving 4. Checked on 100 random arrays.

## 3. Total set bits from 1 to n

Python 3

```
def total_set_bits(n):
    total, p = 0, 1
    while p <= n:
        full, rem = divmod(n + 1, 2 * p)
        total += full * p + max(0, rem - p)
        p *= 2
    return total

print(total_set_bits(10), total_set_bits(100))
```

**Output** (run and checked)

```
17 319
```

**Total number of 1-bits in all numbers from 1 to 100.**

**319**

Each bit position repeats in blocks; the formula counts full blocks and the partial tail. Verified against a direct count for n ≤ 300.

## 4. Maximum XOR of any subset (bitmask)

Python 3

```
def xor_subsets_max(a):
    best = 0
    for mask in range(1 << len(a)):
        x = 0
        for i in range(len(a)):
            if mask >> i & 1: x ^= a[i]
        best = max(best, x)
    return best

print(xor_subsets_max([3, 10, 5, 25, 2, 8]))
```

**Output** (run and checked)

```
31
```

**Maximum XOR over all subsets of [3, 10, 5, 25, 2, 8].**

**31**

All 64 subsets are tried with a bitmask (n ≤ 20 is allowed).

| Signal | Trick |
|---|---|
| “every element appears twice except one” | XOR everything |
| “count the 1s / number of set bits” | n & (n−1) loop or popcount |
| “is it a power of two” | n & (n−1) == 0 |
| “all subsets” with small n | Bitmask loop |
| “swap without extra variable” | a ^= b; b ^= a; a ^= b |
| “divide / multiply by power of 2” | Shifts |

### Shortcuts and tricks

- Use `bin(n).count("1")` or `n.bit_count()` in Python.
- Draw the binary form of a small example before coding.
- Operator precedence: always put parentheses around bit operations (`(n >> i) & 1`).

### Common mistakes

- Using 1 << i with i ≥ 31 in C/Java `int`.
- Forgetting parentheses with comparisons: `n & 1 == 0` parses as `n & (1 == 0)` in C.
- Using bitmask for n = 40 ($2^{40}$ is too large).

### Practice set: Practice

1. Set bits of 13 (1101)? — Basic
2. Is 64 a power of two? (1/0) — Basic
3. XOR of [5,7,5]? — Medium
4. Lowest set bit of 24? — Medium
5. Total set bits 1..16? — Medium
6. Single number in [2,3,5,4,5,3,4]? — Advanced

| | | |
|---|---|---|
| **7.** | Max subset XOR of [1,2,4]? | Advanced |
| **8.** | How many subsets (including empty) does a 10-element set have? | Advanced |

# 12 ADVANCED CODING · REAL PROBLEMS · MUST-DO
# The Real TCS Problems: What They Hide

The best preparation is to look at **real problems**. We studied 7 Advanced Coding problems from TCS campus-hiring tests. Each one is a classic pattern wearing a story. Learn to see the pattern and the story stops mattering. For each problem below: what the story hides, the idea, a full program, and the sample inputs it was run on. The full statements are in the separate “Actual Questions” PDF.

**The 7 real TCS Advanced Coding problems we studied**

| # | Problem | Hidden topic | Level | Status |
|---|---|---|---|---|
| 1 | String Conversion with the Smallest Le | Strings | Hard | verified |
| 2 | Jersey and Board: Beats of the Drum | Math and Number Theory | Medium | verified |
| 3 | Tina and the Perfect Square Cloth | Geometry and Coordinates | Medium | possible reading |
| 4 | Alice and the Verbal Kho-Kho T-shirts | Arrays and Simulation | Easy to Medium | possible reading |
| 5 | The Golden House: Rooms with Exactly K | Subarray Sum and Prefix Sums | Medium | verified |
| 6 | Reddington Restructures His Stocks | Greedy and Sorting | Hard | verified |
| 7 | Radha's Sweet Packing Contest | Hashing, Sets and Simulation | Medium | possible reading |

Two of the seven statements were only partly visible in our photos, so those are marked as readings.

### Honest note

- Problems 4 and 7 were only partly visible in our source photos. Their code follows a **possible reading** that matches the examples we have. Do not treat it as the confirmed official solution.
- Titles, topics and difficulty labels are ours, for study only.

## Problem 1: String Conversion with the Smallest Letter

| Topic | Level | Complexity |
|---|---|---|
| Strings | Hard | O(N) |

Letters can only go down. If any A[i] < B[i], print -1.

Every position with A[i] > B[i] needs a move whose smallest letter is B[i]. That move needs some position currently holding that letter B[i]. Because letters only go down, such a letter must already exist somewhere in A. If it does not exist, print -1.

Moves for different target letters can be done in decreasing order of letter, and every position that needs the same target letter can be fixed together in one move. So the answer is the number of **distinct target letters** that are needed.

Python 3

```
def solve(A, B):
    if any(a < b for a, b in zip(A, B)):
        return -1
    need = {b for a, b in zip(A, B) if a > b}
    if any(t not in set(A) for t in need):
        return -1
    return len(need)

# solve("abcab", "aabab") -> 2 ;  solve("de", "cd") -> -1
n = int(input()); A = input().strip(); B = input().strip()
print(solve(A, B))
```

**Input**

```
2
de
cd
```

**Output** (run) · expected from the statement: **-1** · match

```
-1
```

**Verification:** Checked against both examples and against a brute-force search over all moves on 400 random small strings (no mismatch).

## Problem 2: Jersey and Board: Beats of the Drum

| Topic | Level | Complexity |
|---|---|---|
| Math and Number Theory | Medium | O(N) time, O(N) space |

The board array is a permutation. One beat moves the student at index i to index B[i], so the students move along the cycles of this permutation.

After t beats every student is back home only when t is a multiple of every cycle length. So the answer is the **LCM of all cycle lengths**. For [2 3 1 5 4] the cycles are (1 2 3) and (4 5): LCM(3, 2) = 6.

Python 3

```
from math import gcd

def beats(B):
    n = len(B); seen = [False] * n; ans = 1
    for i in range(n):
        if not seen[i]:
            j, length = i, 0
            while not seen[j]:
                seen[j] = True; j = B[j] - 1; length += 1
            ans = ans * length // gcd(ans, length)
    return ans

# beats([2,3,1,5,4]) -> 6 ;  beats([1,2,3]) -> 1
n = int(input()); B = list(map(int, input().split()))
print(beats(B))
```

**Input**

```
3
1 2 3
```

**Output** (run) · expected from the statement: **1** · match

```
1
```

**Verification:** Checked against both numbers in the statement and against direct simulation of the beats on 300 random permutations (no mismatch). If N is large, the answer can be big; use big integers or the modulus given in the real constraints.

## Problem 3: Tina and the Perfect Square Cloth

| Topic | Level | Complexity |
|---|---|---|
| Geometry and Coordinates | Medium | $O(N^2)$ |

A square has 4 corners. The answer is 4 minus the largest number of given points that can be corners of the same square (at least 1, because any single point can be a corner).

Take every pair of points. If $|dx| = |dy|$ they can be opposite corners; if $dx = 0$ or $dy = 0$ they can be neighbouring corners (two possible squares). For each candidate square count how many of the other two corners are in a hash set. $O(N^2)$ pairs is fine for N up to 1000.

The "bigger cloth" remark only breaks ties between squares that need the same number of added points, so it does not change the printed count.

Python 3

```
def tina(pts):
    S = set(pts); best = 1
    for (x1, y1) in pts:
        for (x2, y2) in pts:
            if (x1, y1) == (x2, y2): continue
            dx, dy = x2 - x1, y2 - y1
            if abs(dx) == abs(dy):                          # opposite corners
                best = max(best, 2 + ((x1, y2) in S) + ((x2, y1) in S))
            if dy == 0:                                     # side along x
                for s in (dx, -dx):
                    best = max(best, 2 + ((x1, y1 + s) in S) + ((x2, y2 + s) in S))
            if dx == 0:                                     # side along y
                for s in (dy, -dy):
                    best = max(best, 2 + ((x1 + s, y1) in S) + ((x2 + s, y2) in S))
    return 4 - best

# Example 1 -> 0 ; Example 2 -> 2
n = int(input()); pts = [tuple(map(int, input().split())) for _ in range(n)]
print(tina(pts))
```

**Input**

```
5
0 0
100 100
200 200
100 0
0 100
```

**Output** (run) · expected from the statement: **0** · match

```
0
```

**Input**

```
3
0 0
2 2
3 3
```

**Output** (run) · expected from the statement: **2** · match

```
2
```

**Verification:** Checked against both examples. The axis-parallel assumption comes from the examples and is not stated in the visible text.

## Problem 4: Alice and the Verbal Kho-Kho T-shirts

| Topic | Level | Complexity |
|---|---|---|
| Arrays and Simulation | Easy to Medium | O(N) |

**Possible reading (not verified against hidden tests):** if the first and the last digit are equal, nobody is penalised: print 0. Otherwise count the friends whose digit differs from the first friend's digit. This gives 0 for Example 1 and 4 for Example 2.

Python 3

```
n = int(input()); D = list(map(int, input().split()))
if D[0] == D[-1]:
    print(0)
else:
    print(sum(1 for x in D if x != D[0]))
```

**Input**

```
3
4 4 4
```

**Output** (run) · expected from the statement: **0** · match

```
0
```

**Input**

```
5
1 2 3 2 2
```

**Output** (run) · expected from the statement: **4** · match

```
4
```

**Verification:** Both examples are consistent with the reading above; no further verification was possible because the full statement is ambiguous.

## Problem 5: The Golden House: Rooms with Exactly K Coins

| Topic | Level | Complexity |
|---|---|---|
| Subarray Sum and Prefix Sums | Medium | O(N) |

Find a contiguous block of rooms whose coins add up to exactly K and whose starting room is the smallest.

If all coin counts are positive, a sliding window (two pointers) works in O(N). If zeros or negative numbers can appear, use prefix sums with a hash map that remembers the first index of every prefix value; scanning from the left still gives the smallest start.

Python 3

```
def rooms(K, a):
    first = {0: 0}            # prefix sum -> smallest prefix length
    s = 0; best = None
    for i, x in enumerate(a, 1):
        s += x
        if s - K in first:
            start = first[s - K] + 1
            if best is None or start < best[0]:
                best = (start, i)
        first.setdefault(s, i)
    return best

# rooms(15, [5,3,7,14,18,1,18,4,8,3]) -> (1, 3)
# rooms(25, [3,4,1,8,9])              -> (1, 5)
n, K = map(int, input().split()); a = list(map(int, input().split()))
r = rooms(K, a)
print(r[0], r[1])
```

**Input**

```
10 15
5 3 7 14 18 1 18 4 8 3
```

**Output** (run) · expected from the statement: **1 3** · match

```
1 3
```

**Input**

```
5 25
3 4 1 8 9
```

**Output** (run) · expected from the statement: **1 5** · match

```
1 5
```

**Verification:** Checked against both examples in the statement and against a brute-force search.

## Problem 6: Reddington Restructures His Stocks

| Topic | Level | Complexity |
|---|---|---|
| Greedy and Sorting | Hard | O(N log N) |

Sort the amounts. Doing t transactions means selling K from the t largest companies and buying K for the t smallest ones (each company is used at most once, and a sale needs amount > K).

For each t from 0 to N/2, the new minimum and maximum come from only a few values: the untouched middle, the sold ones (largest minus K) and the bought ones (smallest plus K). Take the best difference over all t.

Python 3

```
def stocks(a, K):
    a = sorted(a); n = len(a)
    best = a[-1] - a[0]
    for t in range(1, n // 2 + 1):
        if a[n - t] <= K:          # the t-th largest cannot sell K
            break
        mx = max(a[n - t - 1] if n - t - 1 >= t else -1, a[-1] - K, a[t - 1] + K)
        mn = min(a[t] if t <= n - t - 1 else 10**18, a[n - t] - K, a[0] + K)
        best = min(best, mx - mn)
    return best

# stocks([100,1000], 100) -> 700 ; stocks([100,200,300,400,500], 100) -> 200
n, K = map(int, input().split()); a = list(map(int, input().split()))
print(stocks(a, K))
```

**Input**

```
2 100
100 1000
```

**Output** (run) · expected from the statement: **700** · match

```
700
```

**Input**

```
5 100
100 200 300 400 500
```

**Output** (run) · expected from the statement: **200** · match

```
200
```

**Verification:** Checked against both examples and against an exhaustive search over every possible buy/sell choice on 500 random small cases (no mismatch).

## Problem 7: Radha's Sweet Packing Contest

| Topic | Level | Complexity |
|---|---|---|
| Hashing, Sets and Simulation | Medium | O(total boxes) |

**Possible reading (inferred from Example 1):** score = number of boxes + (number of distinct sweet types if the number of distinct types is at least 4, otherwise 0). Print "Radha" if player 1 alone has the highest score, "Tie" if the highest score is shared, otherwise the index (starting from 1) of the single winner.

Python 3

```
def contest(rows):                    # rows[i] = list of box numbers of player i+1
    score = []
    for r in rows:
        k = len(set(r))
        score.append(len(r) + (k if k >= 4 else 0))
    m = max(score)
    if score.count(m) > 1: return "Tie"
    i = score.index(m)
    return "Radha" if i == 0 else i + 1

# contest([[1,2,3,4,5,6],[3,3,3,4,4,4,5,5,5]]) -> "Radha"
n = int(input())
rows = []
for _ in range(n):
    t = list(map(int, input().split()))
    rows.append(t[1:])
print(contest(rows))
```

**Input**

```
2
6 1 2 3 4 5 6
9 3 3 3 4 4 4 5 5 5
```

**Output** (run) · expected from the statement: **Radha** · match

```
Radha
```

**Verification:** Matches Example 1 only. The missing part of the rule could change the result for other inputs, so treat this as a reading of the statement, not a confirmed solution.

## What these seven problems teach

| Story | Pattern | Lesson |
|---|---|---|
| Convert A to B | String comparison, distinct letters | Count what must change; check feasibility first |
| Students move to board positions | Permutation cycles, LCM | Simulate small cases to see the cycle |
| Cloth and coordinates | Geometry with a hash set | Try every pair of points, check the other corners |
| T-shirts of friends | Array reading | Read the statement twice; ambiguity costs marks |
| Rooms with exactly K coins | Prefix sum + hash | First occurrence gives the smallest start |
| Restructure stocks | Sort + greedy over t | Only the extremes matter |
| Sweet packing contest | Set + scoring simulation | Implement the rules exactly as written |

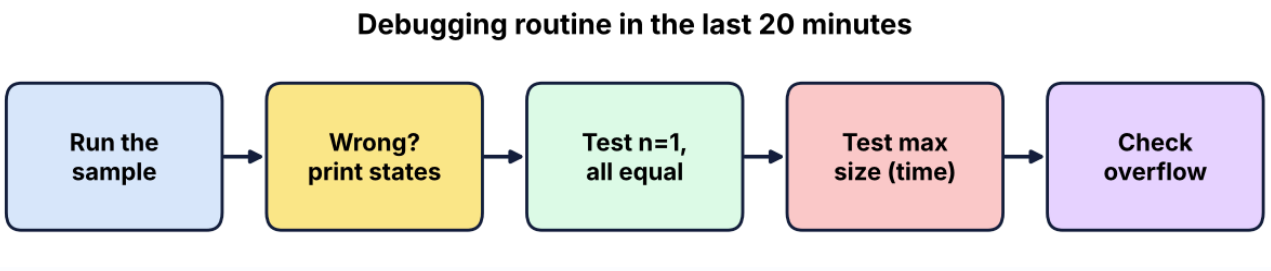
### Shortcuts and tricks

- Simulate the sample by hand first; the pattern usually shows itself.
- Write the simplest brute force first, then compare it with the clever solution on small random cases (what we did for problems 1, 2, 5 and 6).
- If a statement seems ambiguous, choose the reading that matches **all** examples.

ADVANCED CODING · REVISION

# 13 Debugging, Edge Cases, Mini Test and Final Sheet

Most lost marks come from **small mistakes**, not from missing the algorithm. This chapter gives you a debugging routine, a list of edge cases for every topic, a three-problem mini test with verified solutions, and a one-page cheat sheet.




Follow the same routine every time.

## Edge-case checklist

| Topic | Always test |
|---|---|
| Arrays | n = 1, n = 2, all equal, all negative, already sorted, maximum size |
| Strings | Empty or single character, all same letter, uppercase, spaces |
| Prefix sums | K = 0, zeros, subarray starting at index 0 |
| Binary search | Value missing, duplicates, first and last element |
| DP | amount = 0, impossible case, n = 1 |
| Grids | 1 × 1, start or end blocked, no path |
| Math | 0, 1, prime, very large value, mod arithmetic |
| Graphs | Disconnected, single node, repeated edges |
| Output | Exactly one answer per test, no extra spaces |

### Language checklist before you submit

- **C/C++:** `long long` for sums and products; array size fits the constraint; no uninitialised variables.
- **Java:** class name must be `Main` (check the platform); fast input for large n.
- **Python:** recursion limit; read all input quickly; avoid $O(n^2)$ loops over $10^5$ items.
- **All:** print exactly the format asked; remove debug prints.

## Mini test: three problems (90 minutes)

Try each problem yourself first. The solutions below are Python 3 and were run on the sample inputs and compared with a brute-force solver on random small cases.

### Problem A (Medium): Pairs with difference K

Given N integers and K, count pairs of positions (i < j) with $|a[i] - a[j]| = K$. Input: N K, then the array.

Python 3

```
from collections import Counter
def count_pairs(a, k):
    c = Counter(a); total = 0
    for x in c:
        if k == 0:
            total += c[x] * (c[x] - 1) // 2
        elif x + k in c:
            total += c[x] * c[x + k]
    return total
n, k = map(int, input().split())
a = list(map(int, input().split()))
print(count_pairs(a, k))
```

**Input**

```
6 2
1 5 3 4 2 3
```

**Output** (run and checked)

```
5
```

## Problem B (Medium): Minimum jumps

a[i] is the maximum jump length from position i. Find the minimum number of jumps from index 0 to the last index, or −1 if impossible.

Python 3

```
def min_jumps(a):
    n = len(a)
    if n == 1: return 0
    jumps = cur_end = far = 0
    for i in range(n - 1):
        far = max(far, i + a[i])
        if i == cur_end:
            if far <= i: return -1
            jumps += 1; cur_end = far
            if cur_end >= n - 1: break
    return jumps
n = int(input())
a = list(map(int, input().split()))
print(min_jumps(a))
```

**Input**

```
6
2 3 1 1 4 2
```

**Output** (run and checked)

```
3
```

## Problem C (Hard): Grid paths avoiding blocks

A grid has open cells (.) and blocked cells (#). Count the paths from the top-left to the bottom-right moving only right or down, modulo 1,000,000,007.

Python 3

```
MOD = 10**9 + 7
R, C = map(int, input().split())
g = [input().strip() for _ in range(R)]
dp = [[0] * C for _ in range(R)]
dp[0][0] = 1 if g[0][0] == "." else 0
for i in range(R):
    for j in range(C):
        if g[i][j] == "#": dp[i][j] = 0; continue
        if i: dp[i][j] += dp[i-1][j]
        if j: dp[i][j] += dp[i][j-1]
        dp[i][j] %= MOD
print(dp[R-1][C-1])
```

**Input**

```
3 3
...
.#.
...
```

**Output** (run and checked)

```
2
```

**Answers to the three sample inputs.**

**5; 3; 2**

A: the pairs of values with difference 2 are (a[0]=1, a[2]=3), (a[0]=1, a[5]=3), (a[1]=5, a[2]=3), (a[1]=5, a[5]=3), (a[3]=4, a[4]=2); the program counts them with a frequency table, no double loop. B: jump 0→1→4→5 takes 3 jumps (from index 1 with value 3 you can reach index 4, then the last index). C: two paths go around the blocked centre cell.

## Practice set: the 30 problems in the Practice Arena

The Practice Arena that goes with this series contains 30 coding problems with test cases. Use them in this order after finishing each chapter.

| ID | Problem | ID | Problem |
|---|---|---|---|
| p1 | Second Largest Distinct | p16 | 0/1 Knapsack |
| p2 | Balanced Brackets | p17 | Shortest Path in a Maze |
| p3 | Rotate Array | p18 | Next Greater Element |
| p4 | Maximum Subarray Sum | p19 | Count Anagram Groups |
| p5 | Longest Substring Without Repeating Characters | p20 | Longest Palindromic Substring |
| p6 | Count Pairs With Given Sum | p21 | Trapping Rain Water |
| p7 | Count Palindromic Numbers | p22 | Container With Most Water |
| p8 | Spiral Matrix | p23 | First Occurrence in Sorted Array |
| p9 | Longest Increasing Subsequence | p24 | Total Set Bits |
| p10 | Minimum Coins | p25 | Staircase Ways |
| p11 | Subarrays With Sum K | p26 | Minimum Brackets To Add |
| p12 | Minimum Platforms | p27 | Rotate Matrix 90° Clockwise |
| p13 | Number of Islands | p28 | Top K Frequent Elements |
| p14 | Merge Intervals | p29 | Primes In Range |

| | | | |
|---|---|---|---|
| p15 | Word Break | p30 | Edit Distance |


## One-page cheat sheet

| If the problem says ... | Think ... | Time |
|---|---|---|
| subarray sum = K | prefix sum + hash | O(n) |
| longest substring / window | sliding window | O(n) |
| pairs with a sum | sort + two pointers | O(n log n) |
| minimum / maximum, repeated sub-problems | DP | $O(n^2)$ or less |
| count ways | DP counting | — |
| all subsets / permutations, n ≤ 20 | backtracking / bitmask | $O(2^n)$ |
| shortest path in a grid | BFS | O(RC) |
| regions / islands | DFS / BFS | O(RC) |
| weighted shortest path | Dijkstra | O(E log V) |
| groups / connected | DSU | ~O(E) |
| repeat after t steps | cycles + LCM | O(n) |
| next greater | monotonic stack | O(n) |
| single non-repeating | XOR | O(n) |
| minimise the maximum | binary search on answer | O(n log R) |
| intervals | sort + merge | O(n log n) |

### Shortcuts and tricks

- Spend the first 5 minutes reading both problems; start with the one whose pattern you recognise.
- Always keep 10 minutes at the end for edge cases.
- A correct brute force that passes small tests beats an unfinished clever solution.

# Answer key

Answers to every practice set. B = Basic, M = Medium, A = Advanced.

### 2. Arrays

1. 7 (B)
2. -1 (B)
3. 2 (M)
4. 5 (M)
5. 3 (M)
6. 2 (A)
7. 3 (A)
8. 10 (A)

### 3. Strings and hashing

1. 2 (B)
2. 3 (B)
3. 3 (B)
4. 4 (M)
5. bb (M)
6. 2 (M)
7. 1 (A)
8. -1 (A)
9. 3 (A)
10. 5 (A)

### 4. Sorting, search and greedy

1. 2 (B)
2. 1 (B)
3. 1 (M)
4. 1 (M)
5. 6 (M)
6. 3 (A)
7. 3 (A)
8. 4 (A)

### 5. Math and number theory

1. 12 (B)
2. 36 (B)
3. 1 (M)
4. 2 (M)
5. 5 (M)
6. 11 (M)
7. 6 (A)
8. 9 (A)
9. 3 (A)
10. 24 (A)

### 6. Dynamic programming

1. 8 (B)
2. 2 (B)
3. -1 (M)
4. 4 (M)
5. 3 (M)
6. 7 (M)
7. 2 (A)
8. 5 (A)
9. 10946 (A)
10. 3 (A)

### 7. Recursion and backtracking

1. 6 (B)
2. 16 (B)
3. 3 (M)
4. 10 (M)
5. 4 (M)
6. 3 (A)
7. 2 (A)
8. 40 (A)

### 8. Stack and queue

1. 1 (B)
2. 0 (B)
3. 10 (M)
4. 7 (M)
5. 3 (M)
6. 9 (A)
7. 4 (A)
8. 4 (A)

### 9. Grids, BFS and DFS

1. 2 (B)
2. 1 (B)
3. 2 (M)
4. 4 (M)
5. 4 (M)
6. 3 (M)
7. -1 (A)
8. 5 (A)

### 10. Graphs, trees, lists

1. 5 (B)
2. 2 (B)
3. 7 (M)
4. 5 (M)
5. 3 (M)
6. 7 (A)
7. 3 (A)
8. 3 (A)

### 11. Bit manipulation

1. 3 (B)
2. 1 (B)
3. 7 (M)
4. 8 (M)
5. 33 (M)
6. 2 (A)
7. 7 (A)
8. 1024 (A)